



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

高级语言程序设计 High-level Language Programming

Lecture 3 Operators and Constants

Weizheng Zhang (zhangweizheng@hit.edu.cn)
School of Information Science and Technology

Operators

Course Overview

- Basic arithmetic operators

+ - * / %

- Operator precedence

a+b/c

- Assignment operators

= +=

- Increment and decrement

++ --

- Constants

- Marco(before compiling, cautious on use)
- Const

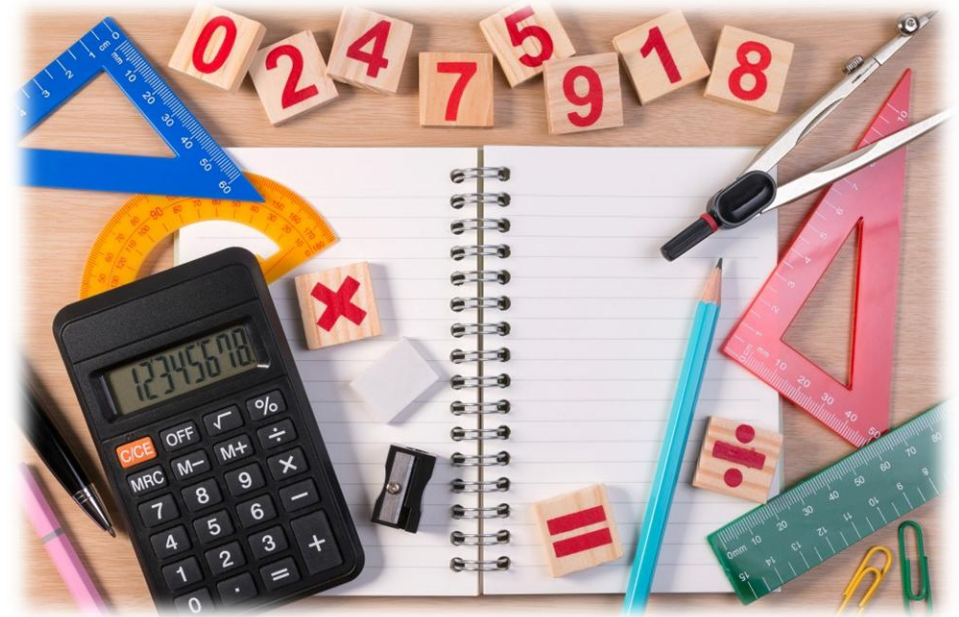
Why we need operators

- Need for computation (heavy work load)
- C++ Operators



Type of operators

- Arithmetic operators
 - Unary arithmetic operators
 - $+$ $-$ $\times$ $/$ $\%$
- Assignment operators
 - Compound-assignment operators
 - $+=$ $-=$ $\times=$ $/=$
- Logical and relational operators
(Lecture 5)
 - $==$ $>$ $<$ $>=$
- Member and pointer operators
(Lecture 9 and 10)
 - `.(access member)` `&(memory address of variable)`



Syntax

- Operator and operand

Example: $2 + 3 = ?$

```
int a = 2, b = 3, c;  
c = a + b;  
           Operator
```

```
int a = 2, b = 3, c;  
c = a + b;  
                        Operand
```

Arithmetic operators

Addition **+**

Subtraction **-**

Multiplication *****

Division **/**

Modulus **%**

Integer type: + - * / %

Floating-point type: + - * /

Character type: treat like integers (ASCII)

Different types have different meanings

Division /

Example: $5 / 2 = ?$

$29/10=?$

```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     double a = 5, b = 2;
6     std::cout << a/b << std::endl;
7
8     return 0;
9 }
```

Ln: 5, Col: 11

[Run](#) [Share](#) Command Line Arguments

2.5

```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     int a = 5, b = 2;
6     std::cout << a/b << std::endl;
7
8     return 0;
9 }
```

Ln: 5, Col: 22

[Run](#) [Share](#) Command Line Arguments

2

Why their results are different?

Division /

Example: $5 / 2 = ?$

Beware of the data type!

```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     double a = 5, b = 2;
6     std::cout << a/b << std::endl;
7
8     return 0;
9 }
```

Floating Division

Ln: 5, Col: 11

[Run](#) [Share](#) Command Line Arguments

2.5

Both operands are floating-point

```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     int a = 5, b = 2;
6     std::cout << a/b << std::endl;
7
8     return 0;
9 }
```

Integer Division

Ln: 5, Col: 22

[Run](#) [Share](#) Command Line Arguments

2

Both operands are integer

Modulus %

- Example of modulo operation

Time format conversion from 24:00H to 12:00H

$$14 \% 12 = (12 + 2) \% 12 = 2$$

Dividend = **Divisor** * **K** + **Remainder** , where **K** must be an integer

14 o'clock becomes 2 o'clock



```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      int a = 14, b = 12;
6      std::cout << a%b << std::endl;
7
8      return 0;
9  }
```

Ln: 6, Col: 35

[Run](#) [Share](#) [Command Line Arguments](#)

2

Modulus %

- Note that the operands of modulus must be **integers**
- Negative operands

$$14 \% 5 = (5 * 2 + 4) \% 5 = 4$$

$$14 \% (-5) = ((-5) * (-2) + 4) \% (-5) = 4$$

$$(-14) \% 5 = (5 * (-2) + (-4)) \% 5 = -4$$

$$A \% B = A - A / B * B$$

$$10 \% 4 = 10 - (\text{int})10 / 4 * 4 = 10 - 8 = 2$$

$$-10 \% 4 = -10 - (\text{int})-10 / 4 * 4 = -10 - (-8) = -2$$

$$-10 \% -4 = -10 - (\text{int})-10 / -4 * (-4) = -10 - (-8) = -2$$

$$10 \% -4 = 10 - (\text{int})10 / -4 * (-4) = 10 - 8 = 2$$

Operator Precedence

- Arithmetic expression containing more than one operators

Example: $5 + 8 / 2 = ?$

Operator Precedence

- Arithmetic expression containing more than one operators

Example: $5 + 8 / 2 = 9$

- Computation order
 - Determined by the order of precedence
 - From left to right for operators with the same precedence

Operator Precedence

C++ Operator Precedence [cppreference.com]

Precedence	Operator	Description
3	+ a	unary plus
	- a	unary minus
5	a * b	multiplication
	a / b	division
	a % b	modulus
6	a + b	addition
	a - b	subtraction
16	=	direct assignment

Precedence	Operator	Description
1	::	Scope resolution
2	a++ a--	Suffix/postfix increment and decrement
	type() type{}	Functional cast
	a()	Function call
	a[] . ->	Subscript Member access
3	++a --a	Prefix increment and decrement
	+ a -a	Unary plus and minus
	! ~	Logical NOT and bitwise NOT
	(type)	C-style cast
	*a	Indirection (dereference)
	&a	Address-of
	sizeof	Size-of ^[note 1]
	co_await	await-expression (C++20)
	new new[]	Dynamic memory allocation
	delete delete[]	Dynamic memory deallocation
	.* ->*	Pointer-to-member
4	.* ->*	Pointer-to-member
5	a * b a / b a % b	Multiplication, division, and remainder
6	a + b a - b	Addition and subtraction
7	<< >>	Bitwise left shift and right shift
8	<=>	Three-way comparison operator (since C++20)
9	< <= > >=	For relational operators < and ≤ and > and ≥ respectively
10	== !=	For equality operators = and ≠ respectively
11	a&b	Bitwise AND
12	^	Bitwise XOR (exclusive or)
13		Bitwise OR (inclusive or)
14	&&	Logical AND
15		Logical OR
16	a?:b:c	Ternary conditional ^[note 2]
	throw	throw operator
	co_yield	yield-expression (C++20)
	=	Direct assignment (provided by default for C++ classes)
	+= -=	Compound assignment by sum and difference
	*= /= %=	Compound assignment by product, quotient, and remainder
17	<<= >>=	Compound assignment by bitwise left shift and right shift
	&= ^= =	Compound assignment by bitwise AND, XOR, and OR
17	,	Comma

Operator Precedence

Examples: $a = ?$

```
int a = 2 + 5 * 2;
```

```
int a = 2 + 2 * 5 % 3;
```

```
int a = - 2 * - 5;
```

```
int a = 2 - - 2 * 5;
```

Precedence	Operator	Description
3	+a	unary plus
	-a	unary minus
5	a*b	multiplication
	a/b	division
	a%b	modulus
6	a+b	addition
	a-b	subtraction
16	=	direct assignment

Operator Precedence

Examples: $a = ?$

```
int a = 2 + 5 * 2;
```

$a = 2 + (5 * 2) = 2 + 10 = 12$

```
int a = 2 + 2 * 5 % 3;
```

$a = 2 + ((2 * 5) \% 3) = 2 + (10 \% 3) = 2 + 1 = 3$

```
int a = - 2 * - 5;
```

$a = (-2) * (-5) = 10$

```
int a = 2 - - 2 * 5;
```

$a = 2 - ((-2) * 5) = 2 - (-10) = 12$

Precedence	Operator	Description
3	+a	unary plus
	-a	unary minus
5	a*b	multiplication
	a/b	division
	a%b	modulus
6	a+b	addition
	a-b	subtraction
16	=	direct assignment

Assignment operators

- Simple assignment

```
int a;  
a = 100;
```

- Multiple assignment

```
int a, b, c;  
a = b = c = 100;  
a = (b = (c = 100)); Associativity: right-to-left
```

- Compound assignment: combine arithmetic and assignment operators together

```
int a = 100;  
a += 5;
```


Compound assignment

- Add operands using simple assignment

```
int a = 100;  
a = a + 5;
```

- Add operands using compound assignment

```
int a = 100;  
a += 5;
```

 Shorthand assignment operator

Compound assignment

- Note that **no space** is allowed between the two symbols of a compound operator (for example, no space between '+' and '=')

Compound operator	Equivalent arithmetic operation
<code>a += b</code>	<code>a = a + b</code>
<code>a -= b</code>	<code>a = a - b</code>
<code>a *= b</code>	<code>a = a * b</code>
<code>a /= b</code>	<code>a = a / b</code>
<code>a %= b</code>	<code>a = a % b</code>

15	<code> </code>	Logical OR
	<code>a?b:c</code>	Ternary conditional ^[note 2]
	<code>throw</code>	throw operator
	<code>co_yield</code>	yield-expression (C++20)
16	<code>=</code>	Direct assignment (provided by default for C++ classes)
	<code>+=</code> <code>-=</code>	Compound assignment by sum and difference
	<code>*=</code> <code>/=</code> <code>%=</code>	Compound assignment by product, quotient, and remainder
	<code><<=</code> <code>>>=</code>	Compound assignment by bitwise left shift and right shift
	<code>&=</code> <code>^=</code> <code> =</code>	Compound assignment by bitwise AND, XOR, and OR
17	<code>,</code>	Comma

Precedence

- Using compound operator to improve code **readability** and **compile efficiency**
- Why?—Reduce memory allocation and unnecessary data duplication**

```
int count = 10;
count = count + 5;
```

```
int count = 10;
count += 5; //
```

Increment and Decrement

- Unary operators
(works on a single operand)

$a = a + 1$

- Increment
 $a++$

$a = a - 1$

- Decrement
 $a--$

Precedence	Operator	Description	Associativity
1	::	Scope resolution	
2	$a++$ $a--$	Suffix/postfix increment and decrement	left-to-right
	$type()$ $type\{\}$	Functional cast	
	$a()$	Function call	
	$a[]$	Subscript	
	$.$ $->$	Member access	
3	$++a$ $--a$	Prefix increment and decrement	
	$+a$ $-a$	Unary plus and minus	
	$!$ $\sim$	Logical NOT and bitwise NOT	
	$(type)$	C-style cast	
	$*a$	Indirection (dereference)	
	$\&a$	Address-of	
	<code>sizeof</code>	Size-of ^[note 1]	
	<code>co_await</code>	await-expression (C++20)	
	<code>new</code> <code>new[]</code>	Dynamic memory allocation	
	<code>delete</code> <code>delete[]</code>	Dynamic memory deallocation	

Increment and Decrement

- Unary operators
(works on a single operand)

`a = a + 1`

- Increment

`a++` `++a`

`a = a - 1`

- Decrement

`a--` `--a`

Precedence	Operator	Description	Associativity
1	::	Scope resolution	
2	a++ a--	Suffix/postfix increment and decrement	left-to-right
	type() type{}	Functional cast	
	a()	Function call	
	a[]	Subscript	
	. ->	Member access	
3	++a --a	Prefix increment and decrement	right-to-left
	+a -a	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	C-style cast	
	*a	Indirection (dereference)	
	&a	Address-of	
	sizeof	Size-of ^[note 1]	
	co_await	await-expression (C++20)	
	new new[]	Dynamic memory allocation	
	delete delete[]	Dynamic memory deallocation	

Increment and Decrement

- Post- and pre-increment
a++ ++a
- Post- and pre-decrement
a-- --a

Value increased/decreased by 1 **after**
the other operations are done

Precedence	Operator	Description
1	::	Scope resolution
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation

Increment and Decrement

- Post- and pre-increment
`a++` `++a`
- Post- and pre-decrement
`a--` `--a`

Value increased/decreased by 1 **before** executing the other operations

Precedence	Operator	Description
1	::	Scope resolution
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation

Increment and Decrement

- Pre-increment

```
int a = 3;  
int b = ++a;
```

a = 4, b = 4

Value of **a** increased by 1 **before** assigning it to **b**

- Post-increment

```
int a = 3;  
int b = a++;
```

a = 4, b = 3

Value of **a** increased by 1 **after** assigning it (originally **a = 3**) to **b**

Precedence	Operator	Description
1	::	Scope resolution
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation

Exercise (Try it yourself!)

```
int a = 2;           a = ?
int b = ++a - 2;     b = ?
```

```
int a = 2;           a = ?
int b = a++ - 2;     b = ?
```

```
int a = 2;           a = ?
int b = a+ + - 2;    b = ?
```

a++ and a--
Value increased/decreased by 1 after
the other operations are done

++a and --a
Value increased/decreased by 1 before
executing the other operations

Precedence	Operator
2	a++
	a--
3	++a
	--a
	+a
	-a
5	a*b
	a/b
	a%b
6	a+b
	a-b
16	=
	+=
	-=
	*=
	/=
	%=

Exercise (Solution)

```
int a = 2;
int b = ++a - 2;
```

$(a = 2+1)$
 $b = (++a) - 2 = 3 - 2 = 1$

```
int a = 2;
int b = a++ - 2;
```

$b = (a++) - 2 = 2 - 2 = 0$
 $a = 3$

```
int a = 2;
int b = a+ + - 2;
```

$b = a + (+(-2)) = a + (-2) = a - 2 = 2 - 2 = 0$
 $a = 2$

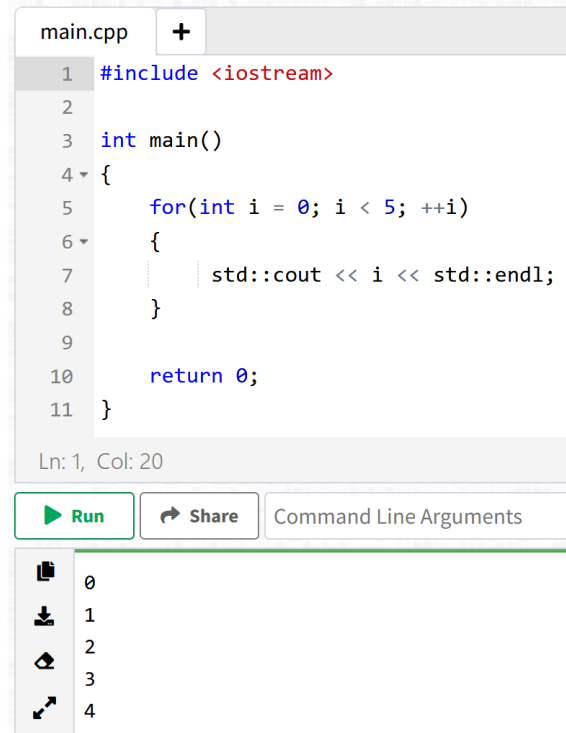
$a++$ and $a--$
Value increased/decreased by 1 after the other operations are done

$++a$ and $--a$
Value increased/decreased by 1 before executing the other operations

Precedence	Operator
2	$a++$
	$a--$
3	$++a$
	$--a$
	$+a$
	$-a$
5	$a*b$
	a/b
	$a\%b$
6	$a+b$
	$a-b$
16	$=$
	$+=$
	$-=$
	$*=$
	$/=$
	$\% =$

Increment and Decrement

- Example usage



```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      for(int i = 0; i < 5; ++i)
6      {
7          std::cout << i << std::endl;
8      }
9
10     return 0;
11 }
```

Ln: 1, Col: 20

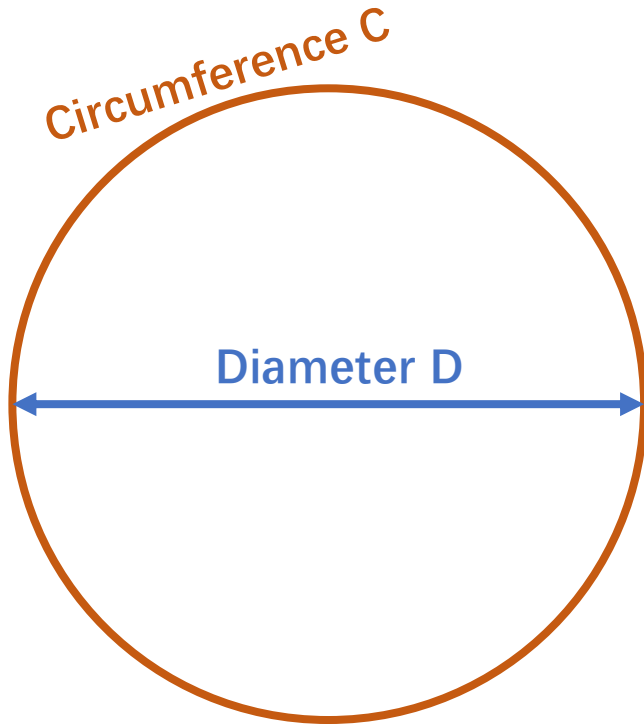
[Run](#) [Share](#)

0
1
2
3
4

- **Avoid** using **more than one increment/decrement operations** in the **same line** of code

Constants

Consider the task: Writing a program that can calculate a tire circumference based on measured diameter



$$C = \pi * D$$

$$\pi = 3.1415926$$



```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << "Circumference of tire 1 = " << 3.1415926 * 19 << std::endl;
6      std::cout << "Circumference of tire 2 = " << 3.1415926 * 24 << std::endl;
7      std::cout << "Circumference of tire 3 = " << 3.1415926 * 28 << std::endl;
8
9      return 0;
10 }
```

Ln: 1, Col: 1

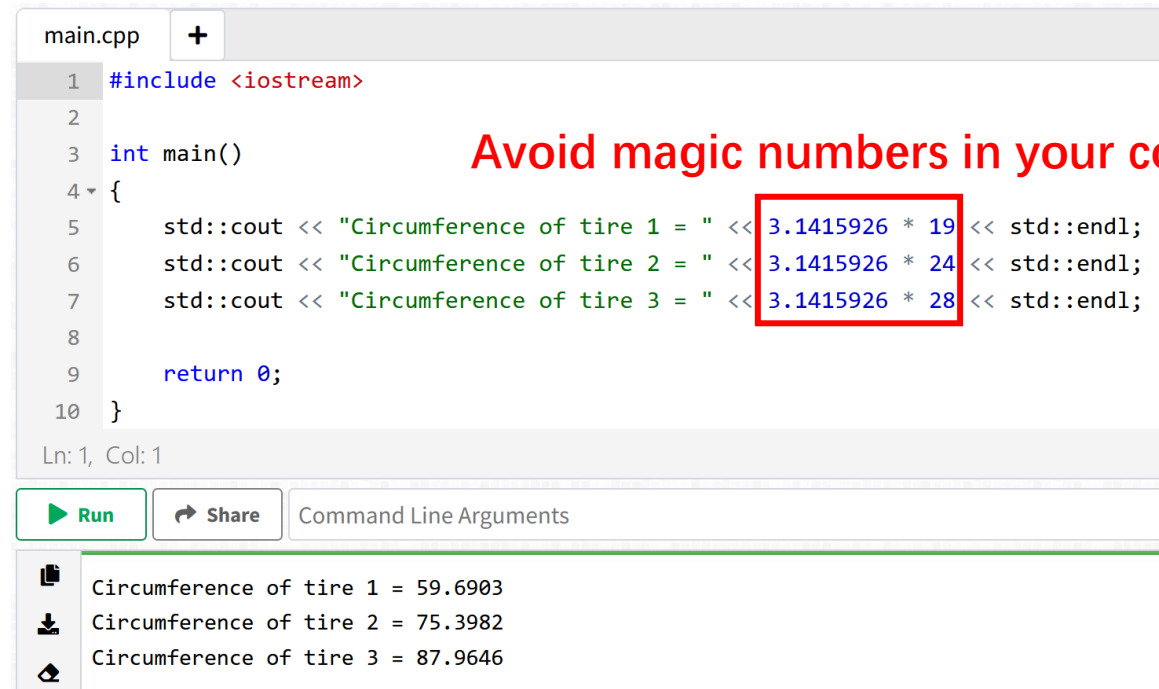
[Run](#) [Share](#) Command Line Arguments

```
Circumference of tire 1 = 59.6903
Circumference of tire 2 = 75.3982
Circumference of tire 3 = 87.9646
```

Constants

- Should avoid creating magic numbers (unspecified numbers in the code)
 - Code readability
 - Prone to mistakes, especially when the numbers need to be changed

How to avoid magic numbers?
Define constants: **marco** or **const variable**



```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "Circumference of tire 1 = " << 3.1415926 * 19 << std::endl;
6     std::cout << "Circumference of tire 2 = " << 3.1415926 * 24 << std::endl;
7     std::cout << "Circumference of tire 3 = " << 3.1415926 * 28 << std::endl;
8
9     return 0;
10 }
Ln: 1, Col: 1
```

Avoid magic numbers in your code!

Run Share Command Line Arguments

```
Circumference of tire 1 = 59.6903
Circumference of tire 2 = 75.3982
Circumference of tire 3 = 87.9646
```

Macro constants

- Syntax

#define MACRO_NAME replacement_value

`#define PI 3.1415926`

- In C++, the preprocessor directives are special commands that are used to instruct the preprocessor. It begins with a '#' symbol and tells the preprocessor to the modify source code **before compilation**.
- When the preprocessor encounters the macro name in our code, it will be replaced by the specified value
- Generally, we capitalize the entire marco name
- No semicolon;

Macro constants

preprocessor replace macro with the specified value

```
main.cpp +
1 #include <iostream>
2
3 #define PI 3.1415926
4 #define D1 19
5 #define D2 24
6 #define D3 28
7
8 int main()
9 {
10     std::cout << "Circumference of tire 1 = " << PI * D1 << std::endl;
11     std::cout << "Circumference of tire 2 = " << PI * D2 << std::endl;
12     std::cout << "Circumference of tire 3 = " << PI * D3 << std::endl;
13
14     return 0;
15 }
```

Ln: 1, Col: 20

[Run](#) [Share](#) Command Line Arguments

```
Circumference of tire 1 = 59.6903
Circumference of tire 2 = 75.3982
Circumference of tire 3 = 87.9646
```

```
main.cpp +
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "Circumference of tire 1 = " << 3.1415926 * 19 << std::endl;
6     std::cout << "Circumference of tire 2 = " << 3.1415926 * 24 << std::endl;
7     std::cout << "Circumference of tire 3 = " << 3.1415926 * 28 << std::endl;
8
9     return 0;
10 }
```

Ln: 1, Col: 1

[Run](#) [Share](#) Command Line Arguments

```
Circumference of tire 1 = 59.6903
Circumference of tire 2 = 75.3982
Circumference of tire 3 = 87.9646
```

Equivalent code

Macro constants

```
main.cpp +
1 #include <iostream>
2
3 #define PI 3.1415926
4 #define D1 19
5 #define D2 24
6 #define D3 28
7
8 int main()
9 {
10     std::cout << "Circumference of tire 1 = " << PI * D1 << std::endl;
11     std::cout << "Circumference of tire 2 = " << PI * D2 << std::endl;
12     std::cout << "Circumference of tire 3 = " << PI * D3 << std::endl;
13
14     return 0;
15 }
```

Ln: 1, Col: 20

Run Share Command Line Arguments

```
Circumference of tire 1 = 59.6903
Circumference of tire 2 = 75.3982
Circumference of tire 3 = 87.9646
```

Where to define them

```
main.cpp +
1 #include <iostream>
2
3 #define PI 3.1415926;
4 #define D1 19;
5 #define D2 24;
6 #define D3 28;
7
8 int main()
```



Must not use semicolons

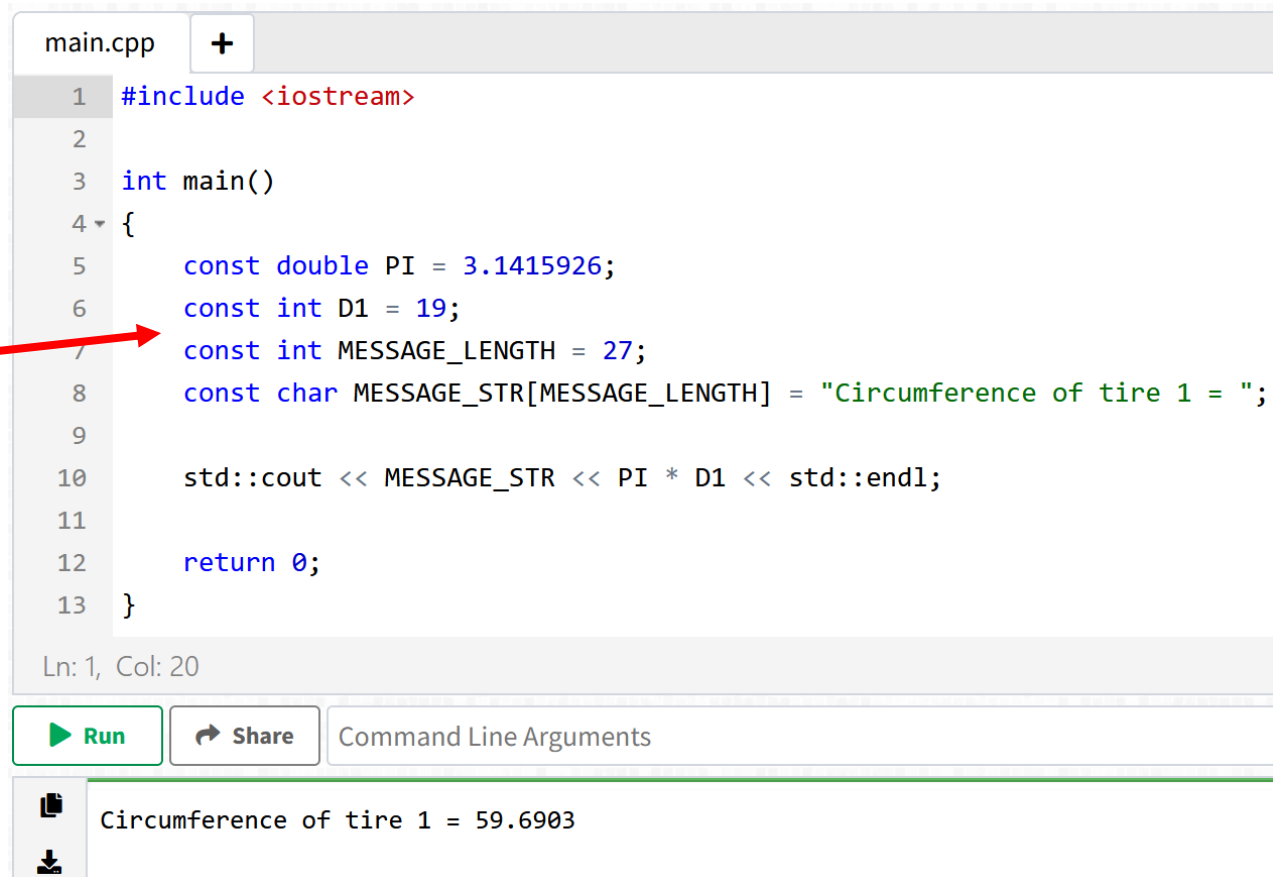
Const variable

- Can use the **const** keyword instead of the `#define` preprocessor directive **to define constant values.**
- The **const** keyword specifies that a variable's value is constant and tells the compiler to prevent the programmer from modifying it

Can define the data type of the constant

Values defined with `const` are subject to type checking

IN C Programming, no `const`. use macro



```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      const double PI = 3.1415926;
6      const int D1 = 19;
7      const int MESSAGE_LENGTH = 27;
8      const char MESSAGE_STR[MESSAGE_LENGTH] = "Circumference of tire 1 = ";
9
10     std::cout << MESSAGE_STR << PI * D1 << std::endl;
11
12     return 0;
13 }
```

Ln: 1, Col: 20

[Run](#) [Share](#) Command Line Arguments

Circumference of tire 1 = 59.6903

Const variable

```
#include <iostream>
using namespace std;
#define N 3 + 2
int main()
{
    int a = 2 * N;
    const int M=3+2;
    int b = 2 * M;
    M=M+1;
    cout<<a<<endl;
    cout<<b;
    return 0;
}
```

```
main.cpp: In function 'int main()':
main.cpp:17:6: error: assignment of read-only variable 'M'
17 |     M=M+1;
   |     ~^~~~
```

Note that you can specify the size of an array with a const variable (or marco)

```
main.cpp +
1  #include <iostream>
2
3  int main()
4  {
5      const double PI = 3.1415926;
6      const int D1 = 19;
7      const int MESSAGE_LENGTH = 27;
8      const char MESSAGE_STR[MESSAGE_LENGTH] = "Circumference of tire 1 = ";
9
10     std::cout << MESSAGE_STR << PI * D1 << std::endl;
11
12     return 0;
13 }
```

Ln: 1, Col: 20



Run



Share

Command Line Arguments



Circumference of tire 1 = 59.6903

```
#include <iostream>
using namespace std;
#define N 3 + 2
int main()
{
    int a = 2 * N;
    const int M=3+2;
    int b = 2 * M;
    cout<<a<<endl;
    cout<<b;
    return 0;
}
```

```
8
10
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Input/Output (I/O)

- Output
 - `std::cout << a_string_constant`
 - `std::cout << a_variable`
 - Concatenate multiple outputs
 - Change line:
`std::endl`
- Input
 - `std::cin >> a_variable`



```
main.cpp +
1  #include<iostream>
2
3
4
5  int main()
6  {
7      int age;
8      std::cout << "Please input your age and press Enter: " << std::endl;
9
10     std::cin >> age;
11
12     std::cout << "Your age is: " << age << std::endl;
13 }
```

Ln: 14, Col: 1

 Run  Share Command Line Arguments

 Please input your age and press Enter:
 12
 Your age is: 12

Precedence	Operator	Description	Associativity
1	<code>a::b</code>	Scope resolution	Left-to-right →
2	<code>a++</code> <code>a--</code> <code>type(a)</code> <code>type{a}</code> <code>a()</code> <code>a[]</code> <code>a.b</code> <code>a->b</code>	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	<code>++a</code> <code>--a</code> <code>+a</code> <code>-a</code> <code>!a</code> <code>~a</code> <code>(type)a</code> <code>*a</code> <code>&a</code> <code>sizeof</code> <code>co_await</code> <code>new - new[]</code> <code>delete - delete[]</code>	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation	Right-to-left ←

4	<code>a.*b</code> <code>a->*b</code>	Pointer-to-member	Left-to-right →
5	<code>a * b</code> <code>a / b</code> <code>a % b</code>	Multiplication, division, and remainder	
6	<code>a + b</code> <code>a - b</code>	Addition and subtraction	
7	<code>a << b</code> <code>a >> b</code>	Bitwise left shift and right shift	
8	<code>a <=> b</code>	Three-way comparison operator (since C++20)	
9	<code>a < b</code> <code>a <= b</code> <code>a > b</code> <code>a >= b</code>	For relational operators < and <= and > and >= respectively	
10	<code>a == b</code> <code>a != b</code>	For equality operators = and != respectively	
11	<code>a & b</code>	Bitwise AND	
12	<code>a ^ b</code>	Bitwise XOR (exclusive or)	
13	<code>a b</code>	Bitwise OR (inclusive or)	
14	<code>a && b</code>	Logical AND	
15	<code>a b</code>	Logical OR	
16	<code>a ? b : c</code> throw co_yield <code>a = b</code> <code>a += b</code> <code>a -= b</code> <code>a *= b</code> <code>a /= b</code> <code>a %= b</code> <code>a <<= b</code> <code>a >>= b</code> <code>a &= b</code> <code>a ^= b</code> <code>a = b</code>	Ternary conditional ^[note 2] throw operator yield-expression (C++20) Direct assignment (provided by default for C++ classes) Compound assignment by sum and difference Compound assignment by product, quotient, and remainder Compound assignment by bitwise left shift and right shift Compound assignment by bitwise AND, XOR, and OR	Right-to-left ←
17	<code>a, b</code>	Comma	Left-to-right →

A Hard Challenge (No grading)

(This challenge is **not** part of the homework! Try it yourself! Not required for any submission)

Exercise (Try it yourself!)

```
int a = 3;  
a += a -= a * -a;
```

a = ?

Precedence	Operator	Associativity
3	+a	right-to-left
	-a	
5	a*b	left-to-right
	a/b	
	a%b	
6	a+b	left-to-right
	a-b	
16	=	right-to-left
	+=	
	-=	
	*=	
	/=	
	%=	

Exercise (Solution)

```
int a = 3;
```

```
a += a -= a * -a; Value stored in a
```

$a += (a -= (a * (-a)))$

$a += (a -= (a * (-3)))$

$a += (a -= (3 * (-3)))$

$a += (a -= (-9))$
 $a += (a = a - (-9))$

$a -= (-9)$ is equivalent
to $a = a - (-9)$

$a += (a = 3 - (-9))$

Value of a is updated! Because
of the '=' operation

$a += 12$
 $a = a + 12$

$a += 12$ is equivalent
to $a = a + 12$

$a = 24$

Value updated!

Precedence	Operator	Associativity
3	+a	right-to-left
	-a	
5	a*b	left-to-right
	a/b	
	a%b	
6	a+b	left-to-right
	a-b	
16	=	right-to-left
	+=	
	-=	
	*=	
	/=	
	%=	

HOMEWORK

Homework 3

- 1. Convert the following mathematical equations into valid C++ statements :

(a) $m = \frac{y_1 - y_2}{x_1 - x_2}$

(b) $y = mx + c$

(c) $a = \frac{b}{c} - \frac{d}{e}$

(d) $C = \frac{5(F - 32)}{9}$

(e) $s = ut + \frac{1}{2}at^2$

- 2. Assuming the following, `int a = 1, b = 2, c = 3 ;` what is the value of a, b and c after each of the following statements?

(a) `a += b ;`

(b) `a /= 3 ;`

(c) `a *= c ;`

(d) `a %= 2 ;`

(e) `a += b+1 ;`

(f) `a += ++b ;`

Homework 3

- 3. Assuming the following, `int a = 12, b = 0, c = 3, d ;` what is the value of a, b, c and d after each of the following statements?

(a) `a++ ;`

(b) `b-- ;`

(c) `d = ++c ;`

(d) `d = c-- ;`

(e) `d = a++ - 2 ;`

(f) `d = a++ + b++ - c-- ;`

Homework 3

- 4. Place parentheses around the following expressions to indicate the order of evaluation :

(a) `a = 1 - 2 * 3 + 4 / 5 ;`

(b) `a = 5 % b + c - d / 10 ;`

(c) `a = ++b * -10 / 5 ;`

- 5. Write a program to compute (a) the volume and (b) the surface area of a box with a height of 10 cm, a length of 11.5 cm, and a width of 2.5 cm.